

Multi-objective Diet Optimization Problem

Bioinspired Algorithms for Optimization

Group Project Assignment

Group Identifier: 11

Members and Contribution

Member	Contribution (%)
Ismael De Los Santos	50%
Gjorgi Andonovski	50%

Professors: Cristian Ramírez Atencia, Ángel Panizo LLedot

Universidad Politécnica de Madrid

May 15, 2026

Contents

1	Problem Formulation	3
1.1	Context and Motivation	3
1.2	Formal Problem Definition	3
1.2.1	Decision Variables and Structure	3
1.2.2	Admissible Food Sets and Search Space	3
1.2.3	Multi-Objective Optimisation Model	3
2	Algorithm Design	4
2.1	Metaheuristics used	4
2.2	Codifications	4
2.3	Constraint handling and a structural equivalence result	5
2.4	Implementation details	5
3	Experiment Design	6
3.1	Parameter settings and tuning protocol	6
3.2	Experimental setup	6
3.3	Data description	7
4	Experimental Results	7
4.1	Aggregated metrics	7
4.2	Statistical significance	8
4.3	Visualisations	9
4.4	Discussion	10
4.5	Code repository	11
5	Conclusion and Future Work	11

Abstract

This work addresses the multi-objective weekly diet planning problem from the Bioinspired Algorithms for Optimization (BAO) course. Given a database of 2,616 real food items and a user profile with a daily caloric target, the goal is to build a seven-day meal plan covering five daily meals (morning snack, breakfast, lunch, afternoon snack and dinner), where each slot may only be filled by foods from the appropriate category. The problem has two conflicting objectives: minimise the total weekly deviation from the caloric target, and minimise the total weekly deviation from a macronutrient split of 50% carbohydrates, 27.5% fat and 22.5% protein.

We model the problem as a constrained two-objective minimisation over a 77-gene weekly chromosome and solve it with four bioinspired metaheuristics covering both course families: **NSGA-II** and **PAES** as evolutionary baselines, and **MOPSO** and **P-ACO** as swarm-intelligence methods. Two encodings are compared, direct integer indices and random keys, both behind a common representation interface. A constraint-handling ablation on NSGA-II direct-index compares *repair*, *penalty* and *death-penalty*; because the domain-aware variator never creates infeasible solutions, the three handlers produce identical quality metrics. From the results section onwards we therefore report NSGA-II direct-index as a single entry and contrast it with the random-key variant, while keeping a separate runtime breakdown for the handlers.

The full benchmark comprises 1,050 runs across seven configurations (30 replicates $\times$ 5 subjects each), evaluated using hypervolume, IGD, Schott's spacing and Deb's Δ -spread. Statistical analysis follows the BAO chapter 6 protocol: Friedman Aligned Ranks for the omnibus and pairwise Wilcoxon signed-ranks with Shaffer's static correction for the post-hoc.

1 Problem Formulation

1.1 Context and Motivation

Diet planning is a common challenge in clinical nutrition, sports science and consumer health. Designing an optimal weekly meal plan requires meeting a personalised daily caloric target T while maintaining a specific macronutrient distribution (50% carbohydrates, 27.5% fats, 22.5% proteins). Balancing these objectives is difficult because improving one can degrade the other — adding a high-protein item may push the caloric total beyond T , while replacing it with a lower-calorie alternative may unbalance the macronutrient ratios.

Constructing feasible plans by hand is impractical given the size of the catalogue and the per-meal category restrictions. From an optimisation standpoint, this is a constrained, combinatorial, multi-objective problem with non-linear macronutrient ratios; exact methods such as Integer Linear Programming are computationally intractable, which makes bioinspired metaheuristics a natural choice because they explore large discrete search spaces efficiently and handle conflicting objectives without gradient information.

1.2 Formal Problem Definition

We formalise the problem through its decision variables, domain constraints and objective functions.

1.2.1 Decision Variables and Structure

A candidate solution is a *weekly meal plan* for a single user, spanning 7 days ($d \in \{1, \dots, 7\}$). Each day consists of five distinct meals, partitioned into specific consumption slots:

- **Morning & Afternoon Snacks:** 1 slot each (fruit or sugar items).
- **Breakfast:** 3 slots (1 drink, 2 solid foods).
- **Lunch & Dinner:** 3 slots each (1 drink, 2 solid foods).

This yields 11 slots per day, totalling 77 slots per weekly plan. A solution is thus represented as a vector $\mathbf{x} \in \mathbb{N}^{77}$, where each decision variable x_j ($j \in \{1, \dots, 77\}$) is the index of the food item assigned to slot j .

1.2.2 Admissible Food Sets and Search Space

Let $\mathcal{F} = \{1, \dots, 2616\}$ denote the complete catalogue of food indices. Foods are categorised via hierarchical group prefixes (e.g. FA for fruits, A for cereals). To ensure culinary and nutritional coherence, each slot j is restricted to a pre-computed *admissible set* $D_j \subseteq \mathcal{F}$, dynamically adjusted based on the user's age:

- **Snacks:** Restricted to fruits (F) and sugars (S).
- **Breakfast Drinks:** Dairy (BA, BH), infusions/powders (PA) and juices (FE, FC).
- **Breakfast Foods:** Cereals (A), eggs (C), fruits (FA) and bacon (MAA), explicitly excluding heavy carbs such as rice, pasta or pizza (AC, AD, AE).
- **Lunch/Dinner Drinks:** General beverages (P, FC, FE); alcoholic drinks (Q) are admitted only if age ≥ 18 .
- **Lunch/Dinner Foods:** Substantive cereals (AC, AD, AE, AF) and any item not classified under breakfast/snack-exclusive categories.

A weekly plan $\mathbf{x}$ is *feasible* iff $x_j \in D_j$ for all j . The feasible search space has size $\prod_{j=1}^{77} |D_j|$; since $|D_j|$ ranges from dozens to thousands of items, the total number of combinations is astronomically large, reinforcing the need for stochastic search.

1.2.3 Multi-Objective Optimisation Model

For a given day d , let $K(\mathbf{x}, d)$ be the total consumed calories and let $C(\mathbf{x}, d)$, $F(\mathbf{x}, d)$, $P(\mathbf{x}, d)$ be the total grams of carbohydrates, fats and proteins. Using Atwater factors, the caloric contribution from each macronutrient is

$$K_C^{(d)} = 4 C(\mathbf{x}, d), \quad K_F^{(d)} = 9 F(\mathbf{x}, d), \quad K_P^{(d)} = 4 P(\mathbf{x}, d). \quad (1)$$

The total daily macronutrient calories are $K_T^{(d)} = K_C^{(d)} + K_F^{(d)} + K_P^{(d)}$, and the percentage contribution of each macronutrient is $p_M^{(d)} = 100 \times K_M^{(d)} / K_T^{(d)}$ for $M \in \{C, F, P\}$. The optimisation problem then reads

$$\text{Minimise } f_1(\mathbf{x}) = \sum_{d=1}^7 |K(\mathbf{x}, d) - T| \quad (2)$$

$$\text{Minimise } f_2(\mathbf{x}) = \sum_{d=1}^7 (|p_C^{(d)} - 50| + |p_F^{(d)} - 27.5| + |p_P^{(d)} - 22.5|) \quad (3)$$

subject to $x_j \in D_j \forall j \in \{1, \dots, 77\}$.

Note on constraints: the original assignment specified a 55% carbohydrate target alongside 27.5% fats and 22.5% proteins. After consultation with the instructors we corrected this to 50% so the three percentages sum to 100%. Consequently, the theoretical optimum for f_2 is zero, attained when the daily proportions match the dietary split exactly.

2 Algorithm Design

2.1 Metaheuristics used

Four metaheuristics span both course families. All four return Pareto archives, share the same feasibility logic, fitness function and metric pipeline, so that any observed difference is attributable to the algorithm itself rather than to evaluation plumbing.

- **NSGA-II** (*evolutionary*) [1]. Selection by non-dominated sorting plus crowding-distance preservation. The de-facto baseline for Pareto-based EAs.
- **PAES** (*evolutionary*) [2]. A (1+1) evolution strategy paired with an adaptive grid archive. Contrasts NSGA-II both in cardinality (single working solution rather than a population) and in selection mechanism (archive-based rather than dominance-based), which makes it a complementary baseline.
- **MOPSO** (*swarm*) [3,5]. Particles move in random-key space, guided by leaders from an external non-dominated archive. The primary leader rule is the Sigma method of Mostaghim and Teich; a random archive pick is available as a fallback. The archive is bounded with crowding-distance pruning.
- **P-ACO** (*swarm*). Ants construct direct-index weekly plans from per-slot pheromone distributions; archive members reinforce the trails without collapsing the objectives into a weighted sum, and crowding-distance pruning preserves diversity.

2.2 Codifications

Two encodings are implemented under a common Representation interface (`diet_bao/representations`). NSGA-II uses both; PAES and P-ACO use direct-index only; MOPSO uses random-key only (required by its velocity update).

- **Direct-index encoding.** A weekly plan is a length-77 integer vector, one gene per slot. Gene j stores the food ID chosen for that slot, and its admissible domain is the precomputed set D_j . The initial generator samples each gene uniformly from D_j . Mutation is *domain-aware*: with probability equal to the mutation rate a gene is re-sampled uniformly from D_j , otherwise it is left unchanged. Crossover is standard n -point crossover; it remains feasible because both parents are feasible and each position can only contain values from its own D_j .
- **Random-key encoding.** A weekly plan is a length-77 real vector $\mathbf{k} \in [0, 1]^{77}$. Decoding maps each k_j to a food in D_j by splitting $[0, 1]$ into $|D_j|$ equal-width bins, so every vector in $[0, 1]^{77}$ decodes to a feasible plan. Mutation uses Gaussian noise ($\sigma = 0.1$) with boundary reflection; crossover again uses the default n -point operator.

2.3 Constraint handling and a structural equivalence result

Three handlers were implemented (repair, penalty, death_penalty) behind a shared ConstraintHandler protocol. The ablation we report applies them to NSGA-II direct-index, the only configuration in our suite where a violation could in principle occur.

Across 150 replicates per handler we observed that the three handlers produce identical hypervolume, IGD, spacing, Δ -spread, best f_1 and best f_2 . The cause is structural: the domain-aware variator never emits an infeasible candidate. Mutation either leaves a gene unchanged (already valid by construction) or re-samples it directly from D_j ; n -point crossover swaps whole segments between two feasible parents and each position is independently constrained to its own D_j . The handlers are still called during evaluation, but because the violation count is always zero they do not change candidate fitness values. As a result, all three search trajectories coincide.

Reporting consequence. Since the three direct-index handlers have the same quality metrics, Section 4 reports NSGA-II as two configurations only: nsga2_di_repair (direct-index) and nsga2_rk_repair (random-key). The penalty and death variants are included only in the runtime breakdown, because their extra feasibility checks are the only observed difference (Table 5). MOPSO and P-ACO carry constraint_handler: "none" because their feasible construction mechanisms do not require a separate handler.

2.4 Implementation details

The project is written in Python 3.10+ and uses inspire for NSGA-II and PAES. MOPSO and P-ACO are native implementations built on the same fitness and archive interfaces as the EA variants. Supporting dependencies are mysql-connector-python for database access, numpy/pandas for data manipulation, scipy.stats for statistical tests and matplotlib for visualisation. The food database is restored from the course-provided MySQL dump and queried through diet_bao.data.

Two optional extensions sit alongside the main benchmark: (i) a multiprocessing execution path (BenchmarkPlan) that distributes the experimental grid across CPU cores without affecting per-run timing; and (ii) a memetic NSGA-II variant that injects a 1-step Lamarckian local search via a memetic_rate parameter. Both are excluded from the main comparison tables to keep them directly comparable.

The codebase is organised around clear responsibilities:

- representations/: direct_index and random_key encodings behind a shared protocol.
- constraints/: repair, penalty and death_penalty handlers behind a shared protocol.
- metrics/: pure-Python implementations of 2D hypervolume, IGD, Schott's spacing and Deb's Δ -spread.
- ea/ and si/: algorithm wrappers (nsga2_diet, paes_diet, mopso_diet, paco_diet).
- experiment/: benchmark composition (AlgorithmConfig, BenchmarkPlan, run_all_subjects) and visualisation helpers.
- stac/: Wilcoxon signed-ranks, Friedman, Friedman Aligned Ranks, Shaffer's post-hoc and Holm correction.

A suite of unit and integration tests is run with pytest before any experiment. Listing 1 shows the domain-aware mutation that drives the equivalence result of Section 2.3.

Listing 1: Domain-aware mutation for direct-index NSGA-II. Because every gene either stays unchanged or is re-sampled from its own D_j , the operator cannot emit an infeasible candidate.

```
1 def variator(random, candidates, args):
2     rate = float(args.get("mutation_rate", 0.1))
3     out = []
4     for cand in candidates:
5         mutant = list(cand)
6         for i, domain in enumerate(state.per_position):
7             if mutant[i] not in domain or random.random() < rate:
8                 mutant[i] = domain[random.randrange(len(domain))]
9         out.append(mutant)
10    return out
```

3 Experiment Design

3.1 Parameter settings and tuning protocol

The notebook implements a deliberate **tuning sweep** on subjects 1 and 4 (one large and one small caloric target), with 5 replicates per cell, selecting configurations by highest mean hypervolume and breaking ties by mean runtime. The grids are:

Algorithm	Swept dimensions	Candidate values
NSGA-II	pop, gen, mut. rate	$\{40, 80\} \times \{40, 80\} \times \{0.05, 0.1, 0.2\}$ (12 cells)
PAES	gen, archive size, mut. rate	$\{400, 800\} \times \{40, 80\} \times \{0.05, 0.1, 0.2\}$ (12 cells)
MOPSO	pop, gen, archive, accel.	$\{30, 60\} \times \{40, 80\} \times \{40, 80\} \times \{1.5, 2.0\}$ (16 cells)
P-ACO	pop, gen, archive, evaporation	$\{30, 60\} \times \{40, 80\} \times \{40, 80\} \times \{0.05, 0.1\}$ (16 cells)

Table 1: Per-algorithm hyperparameter sweep: 56 cells total, replicated 5 times on each of 2 subjects (560 tuning runs).

The full tuning results are saved to `experiments/tuning_results.csv` and the picked winners to `experiments/-tuning_best.csv`. Table 2 reports the actual settings consumed by the full benchmark.

Algorithm	Settings selected	Mean HV (tuning)	Mean rt. (s)
NSGA-II	pop=80, gen=80, mut=0.2	969,654	0.99
PAES	pop=1, gen=800, archive=40, mut=0.1	474,575	1.39
MOPSO	pop=30, gen=80, archive=40, $w = 0.4$, $c_1 = c_2 = 2.0$	489,401	0.21
P-ACO	pop=60, gen=40, archive=40, $\rho = 0.1$, $\alpha = 1$, $\tau_0 = 1$	459,679	16.39

Table 2: Hyperparameter winners from the tuning sweep (`experiments/tuning_best.csv`). PAES uses a (1+1)-ES so its population is 1 by construction; we compensate with a larger generation budget.

Seven main-grid configurations consume the settings in Table 2. Three of them (`nsga2_di_repair`/`penalty`/`death`) form the constraint-handler ablation discussed in Section 2.3; they share every hyperparameter and differ only in the handler. The remaining four span the EA/swarm comparison and the encoding comparison.

Config id	Algorithm	Encoding	Pop	Gen	Notes
<code>nsga2_di_repair</code> [†]	NSGA-II	<code>direct_index</code>	80	80	repair handler, mut. rate 0.2
<code>nsga2_di_penalty</code> [†]	NSGA-II	<code>direct_index</code>	80	80	penalty (1000, 10) per violation
<code>nsga2_di_death</code> [†]	NSGA-II	<code>direct_index</code>	80	80	infeasible $\rightarrow +\infty$
<code>nsga2_rk_repair</code>	NSGA-II	<code>random_key</code>	80	80	Gaussian mutation $\sigma = 0.1$
<code>paes_di_repair</code>	PAES	<code>direct_index</code>	1	800	archive size 40, mut. rate 0.1
<code>mopso_rk</code>	MOPSO	<code>random_key</code>	30	80	Sigma leader, archive 40
<code>paco_di</code>	P-ACO	<code>direct_index</code>	60	40	per-slot pheromones, archive 40

Table 3: Seven main-grid configurations used in the full experiment. [†] The three NSGA-II direct-index handlers produce identical fronts and metrics (Section 2.3); the results section reports them as the single representative entry `nsga2_di_repair`.

3.2 Experimental setup

The seven configurations are executed across the five user profiles with 30 stochastic replicates each, for a total of $7 \times 5 \times 30 = 1,050$ runs. Per-replicate seeds are deterministic and unique (`seed0+r`), making the experiment fully reproducible. The notebook `main.ipynb` schedules every (subject, configuration, seed) triple through `concurrent.futures.ProcessPoolExecutor` (`BenchmarkPlan(n_jobs=4)` by default) and writes the raw rows to `experiments/all_runs.csv`. The parallel execution path only affects wall-clock throughput: each replicate is still timed independently, so the runtime column remains comparable across configurations.

Metrics. For every run we record the best f_1 , the best f_2 , the runtime and the size of the Pareto front. From the raw front we additionally compute four multi-objective indicators:

- **Hypervolume (HV).** 2D area dominated by the front relative to a common per-subject reference point, set from the worst objective values observed across all fronts for that subject with a 1.1x padding plus a small offset. Higher is better.
- **Inverted generational distance (IGD).** Mean Euclidean distance from each point of a per-subject reference set (the union of all fronts produced by all algorithms on that subject) to the nearest point of the obtained front. Lower is better.
- **Schott's spacing.** Standard deviation of consecutive Euclidean distances along the front. Lower is more uniform.
- **Delta Spread (Deb's Δ).** The diversity metric taught in BAO chapter 6.2, combining front extent and uniformity: $\Delta = (\sum_j d_j^e + \sum_i |d_i - \bar{d}|) / (\sum_j d_j^e + |S| \bar{d})$. Lower is better.

Statistical tests. Following the BAO chapter 6 protocol ("Comparing Algorithms Output"), our 30 replicates are paired across configurations because they share the same seed schedule, so we use the two-sided **Wilcoxon signed-ranks test** [9] for 1-vs-1 paired comparisons. The omnibus across configurations uses the **Friedman Aligned Ranks test** [8], a more powerful variant of plain Friedman that ranks all block-aligned observations together. The N-vs-N post-hoc analysis is pairwise Wilcoxon with **Shaffer's static correction** [11]; Holm-Bonferroni [10] is also generated as a conservative companion check.

Limitations.

- Single dataset.** Only the course-provided NutritionPlanning database is used; findings may not transfer to food catalogues with different category structures.
- Degenerate constraint comparison.** Because the direct-index variator (both mutation and n -point crossover) always produces feasible candidates, the three NSGA-II constraint handlers never change the search trajectory and produce identical quality metrics. A meaningful empirical separation would require a variator that can produce infeasible candidates (see Future Work).
- Small tuning sweep.** The hyperparameter grid covers 12–16 cells per algorithm and is evaluated on only two of the five subjects.
- User preferences not used.** The database carries preference, dislike and allergy lists per subject; they are loaded but not yet incorporated into the fitness function or constraints.

3.3 Data description

The MySQL dump provided by the course contains **2,616 food items**, each annotated with a hierarchical food-group prefix and per-item macronutrient content (kcal, carbohydrates, fat and protein, in grams per serving). It also contains **five user profiles** (ids 1–5) covering ages 17, 30, 40, 55 and 72, with caloric targets ranging from 1,401 to 3,103 kcal/day. Each profile additionally carries preference, dislike and allergy lists; these are loaded but not used in the present optimisation. All 2,616 foods participate in the experiments, and all five subjects are used for the comparison.

4 Experimental Results

Result source. The tables and figures in this section are drawn from experiments/all_runs.csv (1,050 rows: 7 configurations $\times$ 5 subjects $\times$ 30 replicates). As established in Section 2.3, the three direct-index NSGA-II handlers produce identical quality metrics across all 150 replicates; from this point onwards we report NSGA-II as two distinct configurations only — direct-index and random-key — yielding five distinct entries in every quality-metric table. A separate breakdown (Table 5) reports the only quantity in which the three handlers differ, namely runtime overhead.

4.1 Aggregated metrics

Table 4 reports the mean of every metric across the 30 replicates and 5 subjects per configuration ($n = 150$ per row), computed from experiments/all_runs.csv. Best value per metric is in bold.

Config	HV $\uparrow$	IGD $\downarrow$	Spacing $\downarrow$	Δ -Spread $\downarrow$	f_1 best $\downarrow$	f_2 best $\downarrow$	Rt. (s) $\downarrow$	Front
nsga2_di_repair [†]	1,014,630	197.13	97.71	0.82	1,343.37	167.77	3.44	80.00
nsga2_rk_repair	1,059,760	152.21	104.68	0.83	1,105.10	167.11	3.62	80.00
paes_di_repair	401,023	469.69	185.66	1.09	1,987.88	123.92	4.20	38.89
mopso_rk	454,137	550.01	345.03	0.90	1,986.80	176.60	2.56	15.37
paco_di	245,278	549.00	362.24	0.80	1,242.08	141.27	337.53	9.02

Table 4: Mean metrics per configuration over $n = 150$ runs (30 replicates $\times$ 5 subjects). Bold marks the best value in each column. [†] Representative of the three NSGA-II direct-index handlers (repair, penalty, death), which produce identical quality metrics (Section 2.3); their runtime differences are reported separately in Table 5.

Handler runtime breakdown. Quality metrics are bit-identical across the three NSGA-II direct-index handlers, but mean runtime is not: penalty and death_penalty still invoke count_violations on every individual even when no violations exist, while repair short-circuits the same check. The overhead is approximately $2\times$ on this workload (Table 5).

Handler	Mean rt. (s)	Std. rt. (s)
nsga2_di_repair	3.44	0.17
nsga2_di_death	6.66	0.21
nsga2_di_penalty	6.70	0.27

Table 5: Runtime breakdown for the three NSGA-II direct-index constraint handlers ($n = 150$ each). All other metrics (HV, IGD, spacing, Δ -spread, f_1 , f_2) are identical across the three rows and therefore omitted; see Table 4 for the shared values.

4.2 Statistical significance

The **Friedman Aligned Ranks** omnibus across all seven configurations rejects the null on every metric ($p < 10^{-70}$ in every case). Table 6 reports the average aligned ranks per configuration on three representative metrics; lower rank means better performance. Since the three NSGA-II direct-index handlers tie exactly on every quality metric (their aligned ranks for HV, IGD, Δ -spread and spacing are identical at 310.61, 343.49, 433.04 and 341.63 respectively), they share a row in the quality columns.

Config	HV rank $\downarrow$	IGD rank $\downarrow$	Runtime rank $\downarrow$
nsga2_rk_repair	326.22	255.52	343.29
nsga2_di_repair [†]	310.61	343.49	316.81
paes_di_repair	787.41	725.01	428.69
mopso_rk	773.72	844.81	212.07
paco_di	859.33	822.69	975.50

Table 6: Friedman Aligned Ranks per metric (lower is better, $n = 150$ aligned blocks per configuration). For hypervolume the aligned observations are sign-flipped so that better HV maps to a lower rank. Omnibus statistics: HV $T = 599.47$, IGD $T = 600.19$, runtime $T = 803.96$; $p < 10^{-70}$ in every case. [†] The HV and IGD ranks are shared with nsga2_di_penalty and nsga2_di_death; on runtime they sit at 702.79 and 699.35 respectively, reflecting the overhead reported in Table 5.

The pairwise **Wilcoxon signed-ranks test with Shaffer’s static correction** on hypervolume yields a clear three-tier ordering:

- **NSGA-II $\gg$ everything else.** Both NSGA-II variants are significantly better on HV than PAES, MOPSO and P-ACO (all adjusted p -values are well below $\alpha = 0.05$).
- **Within NSGA-II:** the random-key variant has the highest mean HV and the best mean IGD, but its HV advantage over direct-index is *not* statistically significant ($p_{\text{raw}} = 0.359$, $p_{\text{adj}} = 0.359$).
- **Constraint handlers:** every pairwise comparison between repair, penalty and death_penalty on HV yields $p_{\text{adj}} = 1.0$ exactly, confirming the structural equivalence of Section 2.3.

- **Swarm methods:** MOPSO is significantly better than P-ACO on HV ($p_{\text{adj}} = 2.51 \times 10^{-11}$), but indistinguishable from PAES ($p_{\text{adj}} = 0.451$). P-ACO is significantly worse than every other method on HV.

4.3 Visualisations

Figure 1 contrasts the five distinct configurations on a single run for subject 1: Pareto fronts (left), best scalar value over generations as a convergence proxy (middle), and front/archive size over generations as a diversity proxy (right). Figure 2 summarises the distribution of every metric across the 150-run sample per configuration. Figure 3 shows the cloud of best (f_1, f_2) values per run on subject 1.

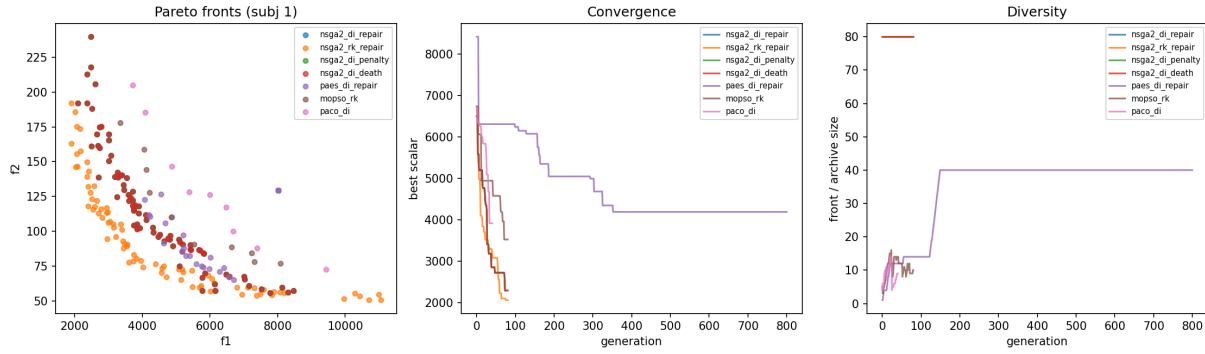


Figure 1: Single-run overview for subject 1: objective space (left), convergence (centre), diversity (right). NSGA-II maintains full 80-point fronts; PAES holds a medium-sized adaptive-grid archive; MOPSO produces a smaller crowding-pruned archive; P-ACO returns a tighter pheromone-reinforced archive. Both NSGA-II penalty and death-penalty are omitted since they overlap exactly with the direct-index repair curve.

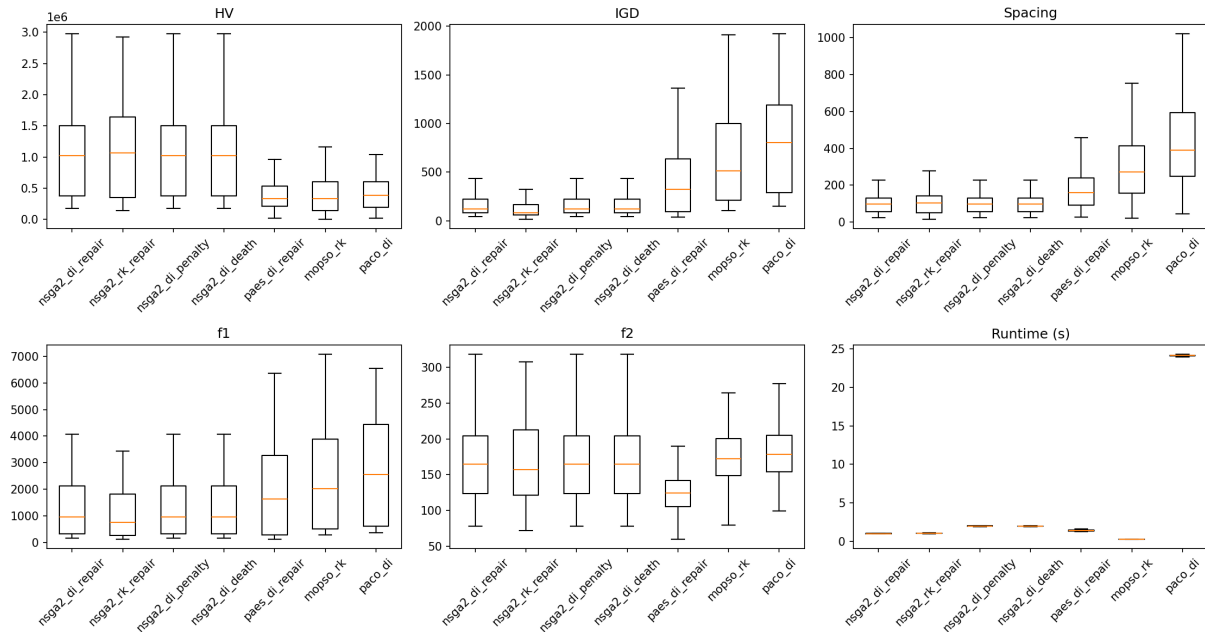


Figure 2: Per-configuration distributions across 30 replicates $\times$ 5 subjects ($n = 150$ per box). Top row: hypervolume, IGD, spacing. Bottom row: best f_1 , best f_2 , runtime in seconds. NSGA-II variants dominate hypervolume and IGD; MOPSO is the fastest method; P-ACO is the slowest in the current implementation. nsga2_di_repair is shown as representative for all three direct-index NSGA-II handlers.

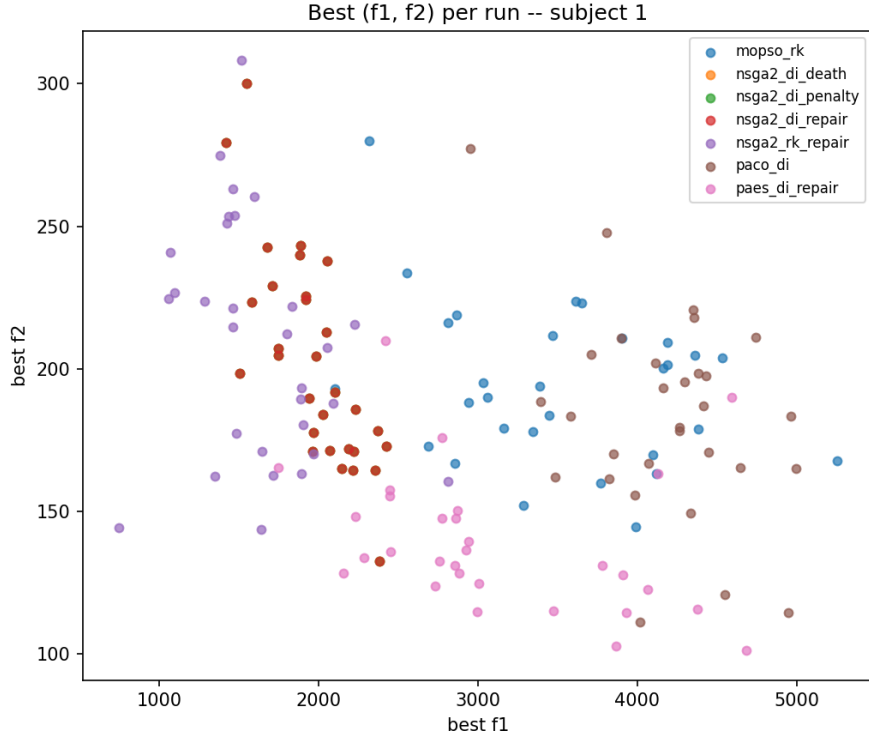


Figure 3: Cloud of best (f_1, f_2) values per run for subject 1, coloured by configuration (30 points per configuration). NSGA-II variants occupy the strongest calorie-error region, while PAES supplies the strongest macronutrient-error points despite weaker Pareto coverage.

4.4 Discussion

Random-key NSGA-II is the strongest overall configuration. Across the full benchmark, `nsga2_rk_repair` leads on hypervolume (1,059,760), IGD (152.21) and best mean f_1 (1,105.10) while remaining within 5% of the fastest NSGA-II variant on runtime (3.62 s vs. 3.44 s). The direct-index variant is a close second on HV (1,014,630) and IGD (197.13), and the HV gap between the two encodings does not reach statistical significance ($p_{\text{adj}} = 0.359$). One possible explanation is that Gaussian perturbations in $[0, 1]^{77}$ explore the decoded categorical space more gradually than discrete uniform re-sampling, while equal-width binning still allows a mutation to move between foods within a slot’s admissible set.

Constraint handlers. The three direct-index handlers (repair, penalty, death_penalty) produce identical hypervolume, IGD, spacing, Δ -spread and best-objective values across all 150 replicates each. Every pairwise Wilcoxon test between them returns $p_{\text{adj}} = 1.0$. The cause is structural rather than statistical: the domain-aware variator cannot produce an infeasible candidate, so the handlers never alter the evaluated fitness. The only measurable cost difference comes from validity checks: repair runs in 3.44 s on average, while penalty (6.70 s) and death_penalty (6.66 s) pay a roughly 2 $\times$ overhead because they still invoke `count_violations` on every individual. This result shows that a feasibility-preserving representation can make explicit constraint handling unnecessary for solution quality, although runtime overhead can remain.

PAES is the best per-point macronutrient solver but the weakest Pareto coverer. `paes_di_repair` records the lowest mean best f_2 across all configurations (123.92), meaning its archive members are individually closest to the prescribed macronutrient split. However, its hypervolume (401,023) is roughly 2.5 $\times$ smaller than NSGA-II’s, reflecting the intrinsic trade-off of a (1+1)-ES: single-trajectory search can exploit one region deeply but cannot match the breadth of a population-based method, even with an enlarged generation budget (800).

Swarm methods diverge sharply. `mopso_rk` is the fastest method in the benchmark (mean runtime 2.56 s) and achieves a mid-range hypervolume (454,137); its HV is indistinguishable from PAES ($p_{\text{adj}} = 0.451$) but

significantly higher than P-ACO. `paco_di`, by contrast, has the weakest HV/runtime trade-off: lowest mean HV (245,278), high IGD (549.00), and a runtime of 337.53 s — roughly 100× MOPSO and 80× NSGA-II direct-index. This suggests that the current pheromone update loop should be profiled and optimised before drawing stronger conclusions about P-ACO itself.

4.5 Code repository

The full source code, tests and experiment notebook required to reproduce every figure and table is available at: <https://github.com/gjorgiandonovski/Multi-objective-Diet-Optimization-Problem>.

5 Conclusion and Future Work

This project tackled the multi-objective weekly diet planning problem with two evolutionary algorithms (NSGA-II, PAES), two swarm-intelligence algorithms (MOPSO, P-ACO), two solution encodings (direct-index and random-key) and three constraint-handling strategies (repair, penalty, death-penalty). The full benchmark covers 1,050 runs across seven configurations (30 replicates × 5 subjects each), evaluated on hypervolume, IGD, Schott’s spacing and Deb’s Δ -spread.

The strongest configuration is `nsga2_rk_repair`, with mean hypervolume 1,059,760 and IGD 152.21 at 3.62 s per run. The direct-index NSGA-II variant is a close second (HV 1,014,630, IGD 197.13, 3.44 s) and the HV gap does not reach statistical significance. PAES delivers the lowest per-plan macronutrient deviation ($f_2 = 123.92$) but the narrowest front (HV 401,023); MOPSO is the fastest method (2.56 s) at mid-range quality; P-ACO has the weakest HV/runtime trade-off and is by far the slowest (337.53 s per run).

The constraint-handler ablation produced a structural finding: because the direct-index variator cannot emit an infeasible candidate, *repair*, *penalty* and *death-penalty* share identical trajectories and identical quality metrics. The only measurable difference is a $\sim 2\times$ runtime overhead in *penalty/death-penalty* from extra validity checks. For this reason, the results use one direct-index NSGA-II quality row plus a separate runtime side-table.

Future work

The following directions are *not yet implemented* and represent natural next steps.

1. Unrestricted variator for a meaningful constraint comparison. The most direct extension is to replace the domain-aware mutation with an operator that can emit infeasible candidates (e.g. sampling each gene from the full catalogue rather than from D_j). This would let *repair*, *penalty* and *death-penalty* diverge in behaviour, turning the present structural equivalence into a genuine empirical comparison.

2. Preference-aware objectives and constraints. The database provides per-subject preference, dislike and allergy lists that are currently loaded but unused. Incorporating them — as a soft penalty for disliked foods or a hard exclusion for allergens — would produce more personalised and practically usable meal plans.

3. Profile and accelerate the P-ACO pheromone loop. P-ACO achieves the worst hypervolume (245,278) at the highest cost (337.53 s per run). This may be caused by an inefficient inner loop in the pheromone update rather than by the algorithm itself; profiling and a vectorised rewrite would clarify this.

4. Denser hyperparameter tuning. The current sweep covers 12–16 cells per algorithm on two of the five subjects. A denser grid on all five subjects would strengthen the conclusions and reduce sensitivity to the tuning sample.

References

1. Deb, K., Pratap, A., Agarwal, S., & Meyarivan, T. (2002). *A fast and elitist multiobjective genetic algorithm: NSGA-II*. IEEE Transactions on Evolutionary Computation, 6(2), 182–197.
2. Knowles, J., & Corne, D. (2000). *Approximating the nondominated front using the Pareto archived evolution strategy*. Evolutionary Computation, 8(2), 149–172.
3. Coello, C. A. C., Pulido, G. T., & Lechuga, M. S. (2004). *Handling multiple objectives with particle swarm optimization*. IEEE Transactions on Evolutionary Computation, 8(3), 256–279.
4. Nebro, A. J., Durillo, J. J., Garcia-Nieto, J., Coello Coello, C. A., Luna, F., & Alba, E. (2009). *SMPSO: A new PSO-based metaheuristic for multi-objective optimization*. IEEE Symposium on Computational Intelligence in Multi-Criteria Decision-Making, 66–73.
5. Kennedy, J., & Eberhart, R. (1995). *Particle swarm optimization*. Proceedings of ICNN'95, vol. IV, 1942–1948.
6. Schott, J. R. (1995). *Fault tolerant design using single and multicriteria genetic algorithm optimization*. Master's thesis, Massachusetts Institute of Technology.
7. Zitzler, E., & Thiele, L. (1999). *Multiobjective evolutionary algorithms: A comparative case study and the strength Pareto approach*. IEEE Transactions on Evolutionary Computation, 3(4), 257–271.
8. Friedman, M. (1937). *The use of ranks to avoid the assumption of normality implicit in the analysis of variance*. Journal of the American Statistical Association, 32(200), 675–701.
9. Wilcoxon, F. (1945). *Individual comparisons by ranking methods*. Biometrics Bulletin, 1(6), 80–83.
10. Holm, S. (1979). *A simple sequentially rejective multiple test procedure*. Scandinavian Journal of Statistics, 6(2), 65–70.
11. Shaffer, J. P. (1986). *Modified sequentially rejective multiple test procedures*. Journal of the American Statistical Association, 81(395), 826–831.
12. Garrett, A. (2012). *Inspyred: Bio-inspired algorithms in Python*. <https://aarongarrett.github.io/inspyred/>
13. Quesada Pajares, J. (2024). *NutritionPlanning: food database and reference scripts for the BAO course*. <https://github.com/javierquesadapajares/NutritionPlanning>
14. Coello, C. A. C., Lamont, G. B., & Van Veldhuizen, D. A. (2007). *Evolutionary Algorithms for Solving Multi-Objective Problems* (2nd ed.). Springer.